

Service: Tipi e Configurazione

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il ruolo del **Service** come astrazione stabile per accedere ai Pod e come viene implementato con iptables/ipvs
 - Distinguere i quattro tipi di Service (ClusterIP, NodePort, LoadBalancer, ExternalName) e scegliere quello appropriato
 - Configurare un Headless Service per StatefulSet e comprendere la risoluzione DNS diretta ai Pod
 - Creare e ispezionare Service tramite **oc expose** e manifest YAML
 - Configurare un Service LoadBalancer su ARO con annotazioni per Azure Load Balancer interno e IP statico
-

Teoria e Concetti Chiave

Il Concetto di Service

I Pod in Kubernetes/OpenShift sono **effimeri**: vengono ricreati con nuovi IP ad ogni restart. Un **Service** risolve questo problema fornendo:

- **VIP stabile** (ClusterIP): un indirizzo IP virtuale che non cambia per tutta la vita del Service
- **Bilanciamento del carico**: distribuisce il traffico tra tutti i Pod che corrispondono al **selector**
- **DNS entry**: ogni Service viene registrato in CoreDNS come **<nome>.<namespace>.svc.cluster.local**
- **Porta stabile**: espone una porta fissa indipendentemente dalle porte dei Pod

Come funziona il forwarding (iptables mode)

```
Client → ClusterIP:porta → iptables DNAT → Pod:targetPort
```

Il componente **kube-proxy** (o **ovnkube-node** con OVN-Kubernetes) scrive regole **iptables** (o flussi OVS) che implementano il NAT dal ClusterIP agli IP dei Pod selezionati.

Tipo 1: ClusterIP (default)

Il tipo **ClusterIP** crea un VIP accessibile **solo dall'interno del cluster**.

Use case: comunicazione service-to-service interna (microservizi, database interni, cache).

```
webapp-pod → ClusterIP:80 → [iptables/OVN] → backend-pod-1:8080  
→ backend-pod-2:8080
```

```
→ backend-pod-3:8080
```

Tipo 2: NodePort

Il tipo **NodePort** espone il Service su una **porta statica su ogni nodo** del cluster (range 30000–32767).

Use case: accesso esterno in ambienti on-premise senza cloud load balancer; testing.

⚠ Attenzione: NodePort non è raccomandato per produzione perché richiede che il client conosca l'IP di un nodo e la porta statica. Usare LoadBalancer o Route per produzione.

```
Client esterno → <IP-nodo>:30080 → [iptables] → ClusterIP:80 → Pod
```

Tipo 3: LoadBalancer

Il tipo **LoadBalancer** richiede al cloud provider di provisionare un **bilanciatore di carico esterno**.

Su ARO:

- OpenShift chiede ad Azure di creare un **Azure Load Balancer** (o Application Gateway con annotazioni specifiche)
- L'IP esterno (pubblico o privato) viene assegnato al campo `status.loadBalancer.ingress[0].ip`
- Per un **LB interno** (privato, solo accessibile nella VNet) si usa l'annotazione Azure

```
Client esterno → Azure LB IP esterno → nodo ARO NodePort → ClusterIP → Pod
```

Tipo 4: ExternalName

Il tipo **ExternalName** crea un **alias DNS CNAME** verso un hostname esterno al cluster. Non crea ClusterIP né endpoints.

Use case: collegare l'applicazione a un database Azure SQL o servizio esterno usando un nome DNS interno al cluster.

```
# Il Service "mydb" risolve a database.corp.example.com
spec:
  type: ExternalName
  externalName: database.corp.example.com
```

Headless Service (clusterIP: None)

Un **Headless Service** non ha ClusterIP: la risoluzione DNS ritorna direttamente gli **indirizzi IP dei singoli Pod**.

Use case: StatefulSet (come visto nel Modulo 4), broker Kafka, cluster Cassandra — dove ogni Pod deve essere raggiungibile individualmente.

```
DNS: myapp-0.myservice.myns.svc.cluster.local → 10.128.4.5 (Pod myapp-0)
DNS: myapp-1.myservice.myns.svc.cluster.local → 10.128.4.6 (Pod myapp-1)
```

selector ed Endpoints

Il Service usa **selector** per individuare i Pod corrispondenti. OpenShift crea automaticamente un oggetto **Endpoints** con gli IP dei Pod selezionati:

```
Service (selector: app=webapp) → Endpoints [10.128.4.5, 10.128.4.6, 10.128.4.7]
```

Endpoint manuale (senza selector): per collegarsi a servizi esterni con IP fisso:

```
# Service senza selector
spec:
  ports:
    - port: 5432

# Endpoint manuale con l'IP del database esterno
kind: Endpoints
subsets:
  - addresses:
    - ip: 192.168.1.100
    ports:
      - port: 5432
```

Sessioni Persistenti: sessionAffinity

sessionAffinity: ClientIP garantisce che le richieste dallo stesso client IP vengano sempre inviate allo stesso Pod:

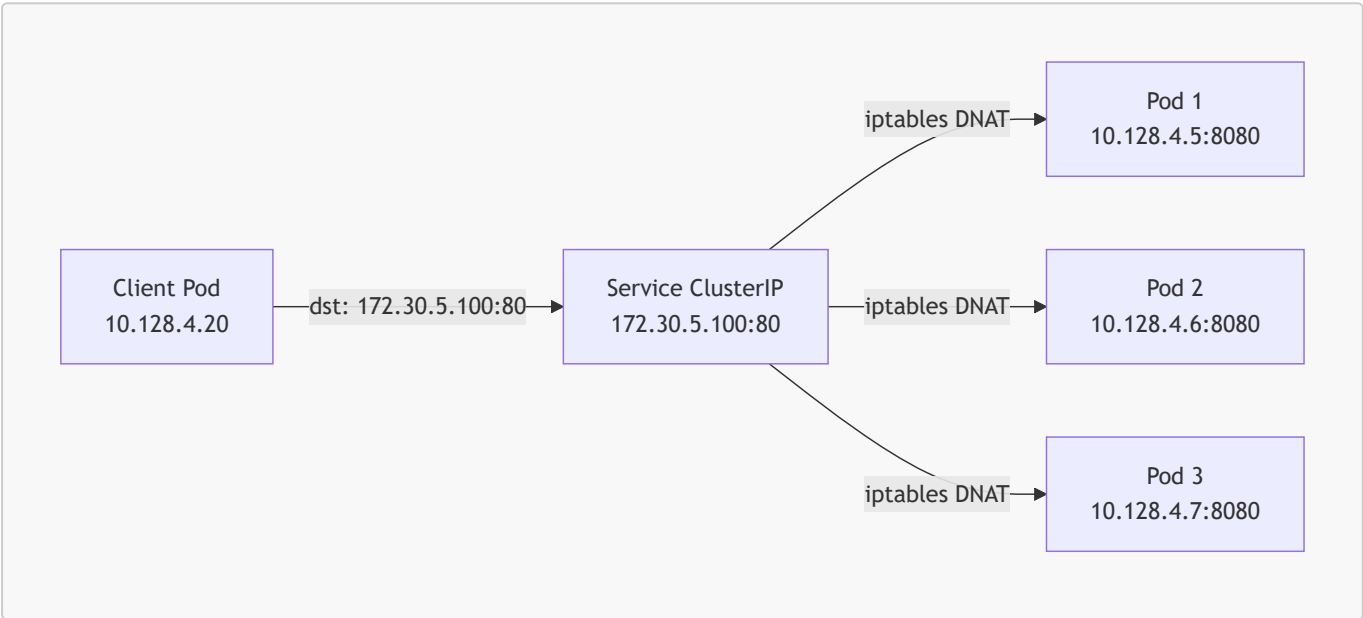
```
spec:
  sessionAffinity: ClientIP
  sessionAffinityConfig:
    clientIP:
      timeoutSeconds: 10800 # ← sessione valida per 3 ore
```

Tabella Comparativa dei Tipi di Service

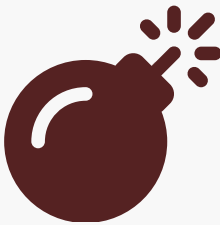
Tipo	Accesso	Caso d'uso	ARO
ClusterIP	Solo interno	Comunicazione tra microservizi	Standard
NodePort	IP nodo + porta 30000-32767	Test, on-premise senza LB	Standard
LoadBalancer	IP esterno pubblico o privato	Accesso esterno produzione	Azure LB
ExternalName	DNS CNAME	Alias per servizi esterni	Standard
Headless (None)	IP Pod diretto via DNS	StatefulSet, cluster DB	Standard

Diagrammi

Flusso ClusterIP: Client → Service → Pod



Service LoadBalancer su ARO



Syntax error in text

mermaid version 11.15.0

Comandi Pratici con Output Atteso

```
# Esporre un Deployment come Service ClusterIP
oc expose deployment webapp --port=80 --target-port=8080
```

```
# Output atteso:
# service/webapp exposed
```

```
# Visualizzare tutti i Service in un namespace
oc get svc -n myproject
```

```
# Output atteso:
# NAME          TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)    AGE
# webapp        ClusterIP      172.30.45.123    <none>           80/TCP      2m
# mysql         ClusterIP      172.30.67.89     <none>           3306/TCP    10d
```

```
# Dettaglio di un Service: selector, endpoints, annotazioni
oc describe svc webapp
```

```
# Output atteso:
# Name:          webapp
# Namespace:     myproject
# Labels:        app=webapp
# Selector:      app=webapp
# Type:          ClusterIP
# IP:            172.30.45.123
# Port:          <unset> 80/TCP
# TargetPort:    8080/TCP
# Endpoints:     10.128.4.5:8080,10.128.4.6:8080,10.128.4.7:8080
# Session Affinity: None
```

```
# Visualizzare gli Endpoints di un Service
oc get endpoints webapp
```

```
# Output atteso:
# NAME          ENDPOINTS                                     AGE
# webapp        10.128.4.5:8080,10.128.4.6:8080,10.128.4.7:8080 5m
```

```
# Creare un Service NodePort da un Deployment
```

```
oc expose deployment webapp \
  --type=NodePort \
  --port=80 \
  --target-port=8080 \
  --name=webapp-nodeport
```

```
# Ottenere la porta assegnata
```

```
oc get svc webapp-nodeport -o jsonpath='{.spec.ports[0].nodePort}'
```

```
# Output atteso: 31205
```

```
# Testare un Service ClusterIP dall'interno del cluster
oc exec -it debug-pod -- curl http://webapp.myproject.svc.cluster.local:80/healthz

# Output atteso:
# {"status":"ok","version":"1.2.3"}
```

```
# Creare un Service LoadBalancer con IP privato ARO (annotazione Azure)
oc apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: webapp-internal
  annotations:
    service.beta.kubernetes.io/azure-load-balancer-internal: "true"
spec:
  type: LoadBalancer
  selector:
    app: webapp
  ports:
    - port: 80
      targetPort: 8080
EOF

# Aspettare l'assegnazione dell'IP interno
oc get svc webapp-internal -w
# Output atteso dopo ~30s:
# NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
# AGE
# webapp-internal    LoadBalancer    172.30.89.12    10.0.1.50        80:31456/TCP
# 35s
```

```
# Visualizzare l'IP del LoadBalancer assegnato da Azure
oc get svc webapp-lb -o jsonpath='{.status.loadBalancer.ingress[0].ip}'

# Output atteso:
# 20.56.123.45
```

Esempi YAML Completi

Service ClusterIP Standard

```
apiVersion: v1
kind: Service
metadata:
```

```

name: webapp
namespace: myproject
labels:
  app: webapp
  tier: frontend
spec:
  type: ClusterIP          # ← tipo default; solo accesso interno al
cluster
  selector:
    app: webapp           # ← seleziona i Pod con questo label
  ports:
    - name: http          # ← nome della porta (opzionale ma consigliato)
      protocol: TCP
      port: 80             # ← porta del Service (ClusterIP)
      targetPort: 8080     # ← porta del container nel Pod
  sessionAffinity: ClientIP # ← sessioni persistenti per IP client
  sessionAffinityConfig:
    clientIP:
      timeoutSeconds: 3600 # ← timeout sessione in secondi (1 ora)

```

Service LoadBalancer Interno per ARO con IP Statico

```

apiVersion: v1
kind: Service
metadata:
  name: webapp-internal-lb
  namespace: myproject
  labels:
    app: webapp
  annotations:
    # ← Rende il LoadBalancer interno (IP privato nella VNet Azure)
    service.beta.kubernetes.io/azure-load-balancer-internal: "true"
    # ← Subnet Azure specifica per il LB interno (opzionale)
    service.beta.kubernetes.io/azure-load-balancer-internal-subnet: "lb-subnet"
    # ← IP statico da assegnare al LB (deve essere nella subnet)
    service.beta.kubernetes.io/azure-load-balancer-ip-address: "10.0.2.100"
spec:
  type: LoadBalancer
  selector:
    app: webapp           # ← seleziona i Pod con questo label
  ports:
    - name: http
      protocol: TCP
      port: 80             # ← porta esterna del LoadBalancer
      targetPort: 8080     # ← porta del container nel Pod
    - name: https
      protocol: TCP
      port: 443
      targetPort: 8443
  # loadBalancerSourceRanges limita l'accesso per CIDR (firewall a livello LB)

```

```
loadBalancerSourceRanges:
- 10.0.0.0/8 # ← solo traffico dalla rete aziendale
```

Headless Service per StatefulSet

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
  namespace: myproject
  labels:
    app: mysql
spec:
  clusterIP: None # ← Headless: nessun VIP assegnato
  selector:
    app: mysql
  ports:
    - name: mysql
      port: 3306
      targetPort: 3306
# Con clusterIP: None, la risoluzione DNS ritorna gli IP dei singoli Pod:
# mysql-0.mysql-headless.myproject.svc.cluster.local → IP Pod mysql-0
# mysql-1.mysql-headless.myproject.svc.cluster.local → IP Pod mysql-1
```

Note Specifiche per Azure ARO

Annotazioni Azure per Service LoadBalancer

ARO supporta le seguenti annotazioni per personalizzare il comportamento del LoadBalancer Azure:

Annotazione	Descrizione	Valore
<code>azure-load-balancer-internal</code>	LB interno (IP privato VNet)	<code>"true"</code>
<code>azure-load-balancer-internal-subnet</code>	Subnet Azure per LB interno	nome subnet
<code>azure-load-balancer-ip-address</code>	IP statico per il LB	IP nella subnet
<code>azure-load-balancer-resource-group</code>	Resource Group dell'IP pubblico	nome RG
<code>azure-dns-label-name</code>	DNS label per l'IP pubblico Azure	label DNS

Provisioning del LoadBalancer Azure

Quando si crea un Service di tipo LoadBalancer su ARO, OpenShift Cloud Controller Manager:

1. Chiede ad Azure di creare un Azure Load Balancer nella resource group del cluster
2. Crea un frontend IP configuration e una backend pool con i nodi worker
3. Aggiorna `service.status.loadBalancer.ingress` con l'IP assegnato

⚠ Attenzione: La creazione di un Azure Load Balancer può richiedere fino a 60-120 secondi. Usare `oc get svc -w` per attendere l'assegnazione dell'IP.

Limitazioni Service NodePort su ARO

Su ARO, i NSG Azure dei worker node permettono il traffico NodePort solo se configurati esplicitamente. Di default, i NodePort non sono accessibili dall'esterno della VNet.

```
# Verificare le regole NSG per i NodePort
az network nsg rule list \
  --resource-group aro-cluster-rg \
  --nsg-name aro-nsg-worker \
  --query "[?destinationPortRange>='30000' && destinationPortRange<='32767']" \
  --output table
```

Riepilogo dei Punti Chiave

- Il **Service** fornisce un VIP stabile e un DNS entry per accedere a un gruppo di Pod con selector
- **ClusterIP** è il tipo default (solo interno); **NodePort** espone su porta statica su ogni nodo; **LoadBalancer** provisiona un LB cloud; **ExternalName** è un CNAME DNS
- Gli **Endpoints** sono l'oggetto che traccia gli IP dei Pod selezionati dal Service
- Un Headless Service (**clusterIP: None**) risolve il DNS direttamente agli IP dei Pod, necessario per StatefulSet
- Su ARO, il LoadBalancer interno si ottiene con l'annotazione **azure-load-balancer-internal: "true"**
- **loadBalancerSourceRanges** permette di restringere l'accesso al LB per CIDR (firewall lato Azure LB)

Domande di Verifica

1. Un microservizio **payment-service deve essere raggiungibile solo da altri Pod nel cluster, non dall'esterno. Quale tipo di Service è corretto?**

- A) LoadBalancer
- B) NodePort
- C) ClusterIP ☒
- D) ExternalName

2. Un cluster ARO deve esporre un'API REST con IP privato accessibile solo dalla VNet aziendale. Quale combinazione è corretta?

- A) Service di tipo **ClusterIP** + Route OpenShift
- B) Service di tipo **LoadBalancer** con annotazione **azure-load-balancer-internal: "true"** ☒
- C) Service di tipo **NodePort** con regola NSG Azure
- D) Service di tipo **ExternalName** puntando all'IP privato

3. Un StatefulSet `kafka` ha 3 replica. Quale tipo di Service permette di raggiungere `kafka-0` direttamente per nome DNS?

- A) Service ClusterIP con selector `statefulset.kubernetes.io/pod-name: kafka-0`
- B) Service NodePort con porte diverse per ogni Pod
- C) Headless Service con `clusterIP: None` 
- D) Service ExternalName con alias `kafka-0`

4. `oc get endpoints webapp` ritorna una lista vuota. Qual è la causa più probabile?

- A) Il Service non è di tipo ClusterIP
- B) Il campo `targetPort` del Service è errato
- C) Nessun Pod corrisponde al `selector` del Service 
- D) Il namespace del Service è diverso da quello dei Pod

5. Quale comando mostra l'IP esterno assegnato a un Service di tipo LoadBalancer su ARO?

- A) `oc describe svc webapp-lb | grep "External IP"`
- B) `oc get svc webapp-lb -o jsonpath='{.status.loadBalancer.ingress[0].ip}'` 
- C) `oc get endpoints webapp-lb -o wide`
- D) `az network lb show --name webapp-lb`